

```
from abc import ABC, abstractmethod
```

```
# =====
```

```
# Продукт - Компьютер
```

```
# =====
```

```
class Computer:
```

```
    def __init__(self):
```

```
        self.processor = None
```

```
        self.ram = None
```

```
        self.os = None
```

```
    def set_processor(self, processor: str):
```

```
        self.processor = processor
```

```
    def set_ram(self, ram: str):
```

```
        self.ram = ram
```

```
    def set_os(self, os: str):
```

```
        self.os = os
```

```
    def show_info(self):
```

```
        print("Характеристики компьютера:")
```

```
        print(f"Процессор: {self.processor}")
```

```
        print(f"Оперативная память: {self.ram}")
```

```
        print(f"Операционная система: {self.os}")
```

```
        print("-" * 40)
```

```
# =====
```

```
# Builder - интерфейс разработчика компьютеров
```

```
# =====
```

```
class IComputerDeveloper(ABC):
```

```
    @abstractmethod
```

```
    def install_processor(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def install_ram(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def install_os(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def get_computer(self) -> Computer:
```

```
        pass
```

```

# =====
# ConcreteBuilder - разработчик HP
# =====
class HPComputerDeveloper(ComputerDeveloper):
    def __init__(self):
        self.computer = Computer()

    def install_processor(self):
        self.computer.set_processor("Intel Core i5")

    def install_ram(self):
        self.computer.set_ram("16 GB")

    def install_os(self):
        self.computer.set_os("Windows 11")

    def get_computer(self) -> Computer:
        return self.computer

# =====
# ConcreteBuilder - разработчик DELL
# =====
class DELLComputerDeveloper(ComputerDeveloper):
    def __init__(self):
        self.computer = Computer()

    def install_processor(self):
        self.computer.set_processor("AMD Ryzen 7")

    def install_ram(self):
        self.computer.set_ram("32 GB")

    def install_os(self):
        self.computer.set_os("Ubuntu 22.04")

    def get_computer(self) -> Computer:
        return self.computer

# =====
# Director - директор завода
# =====
class Director:
    def __init__(self, developer: ComputerDeveloper):
        self.developer = developer

```

```

def build_full_computer(self):
    self.developer.install_processor()
    self.developer.install_ram()
    self.developer.install_os()

def build_computer_without_os(self):
    self.developer.install_processor()
    self.developer.install_ram()

# =====
# Клиентский код
# =====
if __name__ == "__main__":
    print("Сборка компьютера HP:")
    hp_developer = HPComputerDeveloper()
    director = Director(hp_developer)
    director.build_full_computer()
    hp_computer = hp_developer.get_computer()
    hp_computer.show_info()

    print("Сборка компьютера DELL без операционной системы:")
    dell_developer = DELLComputerDeveloper()
    director = Director(dell_developer)
    director.build_computer_without_os()
    dell_computer = dell_developer.get_computer()
    dell_computer.show_info()

```

Сборка компьютера HP:
 Характеристики компьютера:
 Процессор: Intel Core i5
 Оперативная память: 16 GB
 Операционная система: Windows 11

Сборка компьютера DELL без операционной системы:
 Характеристики компьютера:
 Процессор: AMD Ryzen 7
 Оперативная память: 32 GB
 Операционная система: None

Введение

В процессе разработки программного обеспечения часто возникает необходимость создавать сложные объекты, состоящие из множества частей. При этом важно, чтобы процесс создания был гибким, управляемым и не зависел от конкретных реализаций.

Для решения подобных задач используется шаблон проектирования **Строитель (Builder)**. В данном практическом задании была реализована программа на языке Python, демонстрирующая применение этого паттерна на примере сборки компьютеров различных конфигураций.

Основная идея работы — разделить процесс создания объекта и его представление, что позволяет создавать различные варианты объектов, используя один и тот же процесс сборки.

Теоретическая часть

Паттерн **Builder (Строитель)** — это порождающий шаблон проектирования, который позволяет создавать сложные объекты пошагово. Он отделяет процесс конструирования объекта от его представления, благодаря чему один и тот же процесс может создавать разные представления объекта.

Основные компоненты паттерна:

- **Product (Продукт)** — создаваемый объект (в данном случае компьютер);
- **Builder (Строитель)** — интерфейс, определяющий этапы создания объекта;
- **Concrete Builder (Конкретный строитель)** — реализует этапы создания;
- **Director (Директор)** — управляет процессом создания объекта.

Преимущества паттерна:

- возможность пошагового создания объекта;
- гибкость в конфигурации создаваемых объектов;
- переиспользование кода;
- удобство добавления новых вариантов объектов.

Недостатки:

- увеличение количества классов;
- усложнение структуры программы.

Заключение

В ходе выполнения практической работы был изучен и реализован паттерн «Строитель». Данный паттерн позволяет создавать сложные объекты поэтапно, обеспечивая гибкость и удобство настройки.

Практическая реализация показала, что использование Builder значительно упрощает процесс создания объектов с различными конфигурациями и улучшает архитектуру программы.

Полученные знания могут быть применены при разработке сложных систем, где требуется поэтапное создание объектов и высокая гибкость.